

STAT534 HW5

Problem 1: Marginal Log Likelihood

Part A: Solution

```
#include "matrices.h"
#include <lapacke.h>

// log marginal likelihood function
double marglik(int n, int p, double **data, int lenA, int *A)
{
    int i, j;

    // 1. define variables
    double **D1 = allocmatrix(n, 1);
    double **DA = allocmatrix(n, lenA);

    // response Y
    for (i = 0; i < n; i++)
    {
        D1[i][0] = data[i][0];
    }
    // data X
    for (j = 0; j < lenA; j++)
    {
        for (int i = 0; i < n; i++)
        {
            DA[i][j] = data[i][A[j] - 1];
        }
    }

    // debugmatrix(n, 1, D1);
    // printf("\n");
    // debugmatrix(n, lenA, DA);

    // 2. compute MA = I + DA^T DA
    double **DAt = transposematrix(n, lenA, DA);
    double **MA = allocmatrix(lenA, lenA);

    matrixproduct(lenA, n, lenA, DAt, DA, MA);

    for (i = 0; i < lenA; i++)
    {
        MA[i][i] += 1.0;
    }
}
```

```

double log_det_MA = logdet(lenA, MA);

// 3. compute D1^T D1
double d1td1 = 0.0;
for (i = 0; i < n; i++)
{
    d1td1 += D1[i][0] * D1[i][0];
}

// 4. compute D1^T DA MA^-1 DA^T D1
double **b = allocmatrix(lenA, 1);
matrixproduct(lenA, n, 1, DAt, D1, b);
// get MA^-1
inverse(lenA, MA);

double **temp = allocmatrix(lenA, 1);
matrixproduct(lenA, lenA, 1, MA, b, temp);

double prod = 0.0;
for (i = 0; i < lenA; i++)
{
    prod += b[i][0] * temp[i][0];
}

// 5. calc. log marginal lik
double prior = (lenA + 2.0) / 2.0;
double post = (n + lenA + 2.0) / 2.0;

double q = 1.0 + d1td1 - prod;

double log_marlik = (lgamma(post) - lgamma(prior) - 0.5 * log_det_MA - post * log(q));

// free mem
freematrix(n, D1);
freematrix(n, DA);
freematrix(lenA, DAt);
freematrix(lenA, MA);
freematrix(lenA, b);
freematrix(lenA, temp);

return (log_marlik);
}

```

Part B: AI Solution

i. Prompt

The data file `erdata.txt` has $n = 158$ samples and $p = 51$ variables. Column 1 is the response variable and the remaining columns are explanatory variables.

Write the C/C++ function:

```
double marglik(int n, int p, double** data, int lenA, int* A)
```

that calculates the log marginal likelihood for regression $[1|A]$ using $M_A = I + D_A^T D_A$ and the formula in the

homework PDF. Use the provided matrix helper functions such as allocmatrix, freematrix, transposematrix, matrixproduct, inverse, and logdet.

ii. Code Generated

```
double marglik(int n, int p, double **data, int lenA, int *A)
{
    int i, j;
    double **XA = allocmatrix(n, lenA);
    double **XAT = NULL;
    double **MA = NULL;
    double **rhs = NULL;
    double **sol = NULL;

    (void)p;

    for (i = 0; i < n; i++)
    {
        for (j = 0; j < lenA; j++)
        {
            XA[i][j] = data[i][A[j] - 1];
        }
    }

    XAT = transposematrix(n, lenA, XA);
    MA = allocmatrix(lenA, lenA);
    matrixproduct(lenA, n, lenA, XAT, XA, MA);

    for (i = 0; i < lenA; i++)
    {
        MA[i][i] += 1.0;
    }

    double logdetMA = logdet(lenA, MA);
    double yty = 0.0;
    for (i = 0; i < n; i++)
    {
        yty += data[i][0] * data[i][0];
    }

    rhs = allocmatrix(lenA, 1);
    for (i = 0; i < lenA; i++)
    {
        for (j = 0; j < n; j++)
        {
            rhs[i][0] += XAT[i][j] * data[j][0];
        }
    }

    inverse(lenA, MA);
    sol = allocmatrix(lenA, 1);
    matrixproduct(lenA, lenA, 1, MA, rhs, sol);

    double quad = 0.0;
```

```

for (i = 0; i < lenA; i++)
{
    quad += rhs[i][0] * sol[i][0];
}

double alpha0 = (lenA + 2.0) / 2.0;
double alphaN = (n + lenA + 2.0) / 2.0;
double scale = 1.0 + yty - quad;
double out = lgamma(alphaN) - lgamma(alpha0) - 0.5 * logdetMA
            - alphaN * log(scale);

freematrix(n, XA);
freematrix(lenA, XAT);
freematrix(lenA, MA);
freematrix(lenA, rhs);
freematrix(lenA, sol);

return out;
}

```

Part C: Comparison

- **Overall**
 - The AI solution is correct and understandable.
 - My solution is more directly tied to the homework notation and was tested in the full program.
- **Correctness**
 - The AI code implements the same marginal likelihood formula as my code.
 - It correctly uses $A[j] - 1$ to convert the one-based variable indices to zero-based C++ indices.
 - It computes $M_A = I + D_A^T D_A$, the log determinant, and the quadratic form correctly.
 - P.S. Since the AI code follows the same formula and uses the same helper functions, the result should match.
- **Readability**
 - The AI code is readable and uses clear variable names such as `XA`, `XAT`, `rhs`, and `sol`.
 - My solution is slightly closer to the notation in the homework because it explicitly creates `D1` and `DA`.
 - The AI code is a little shorter because it reads the response directly from `data[i][0]`, which I think is more efficient yet less readable.
- **Data Structures**
 - Both codes use `double**` matrices.
 - Both codes allocate matrices for the selected predictors, transpose, M_A , and temporary vectors.
 - My code stores the response as a separate one-column matrix, while the AI code does not.
- **Algorithms**
 - Both codes use direct matrix multiplication, matrix inversion, and log determinant computation.
 - Both codes explicitly compute M_A^{-1} instead of solving a linear system. (which is suggested by the AI)

Problem 2: GSL version

Part A: Solution

```
#include "matrices.h"

// log marginal likelihood w/ GSL
double marglik(gsl_matrix *data, int lenA, int *A)
{
    int i, j;
    int n = data->size1;

    // 1. define variables
    gsl_matrix *D1 = gsl_matrix_alloc(n, 1);
    gsl_matrix *DA = gsl_matrix_alloc(n, lenA);

    // response Y
    for (i = 0; i < n; i++)
    {
        gsl_matrix_set(D1, i, 0, gsl_matrix_get(data, i, 0));
    }
    // data X
    for (j = 0; j < lenA; j++)
    {
        for (i = 0; i < n; i++)
        {
            gsl_matrix_set(DA, i, j, gsl_matrix_get(data, i, A[j] - 1));
        }
    }

    // 2. compute MA = I + DA^T DA
    gsl_matrix *DA_t = transposematrix(DA);
    gsl_matrix *MA = gsl_matrix_alloc(lenA, lenA);

    matrixproduct(DA_t, DA, MA);

    for (i = 0; i < lenA; i++)
    {
        gsl_matrix_set(MA, i, i, gsl_matrix_get(MA, i, i) + 1.0);
    }

    double log_det_MA = logdet(MA);

    // 3. compute D1^T D1
    double d1td1 = 0.0;
    for (i = 0; i < n; i++)
    {
        d1td1 += gsl_matrix_get(D1, i, 0) * gsl_matrix_get(D1, i, 0);
    }

    // 4. compute D1^T DA MA^-1 DA^T D1
    gsl_matrix *b = gsl_matrix_alloc(lenA, 1);
    matrixproduct(DA_t, D1, b);
```

```

gsl_matrix *MA_inv = inverse(MA);
gsl_matrix *temp = gsl_matrix_alloc(lenA, 1);
matrixproduct(MA_inv, b, temp);

double prod = 0.0;
for (i = 0; i < lenA; i++)
{
    prod += gsl_matrix_get(b, i, 0) * gsl_matrix_get(temp, i, 0);
}

// 5. calc. log marignal lik
double prior = (lenA + 2.0) / 2.0;
double post = (n + lenA + 2.0) / 2.0;
double q = 1.0 + d1td1 - prod;

double log_marlik = (lgamma(post) - lgamma(prior) - 0.5 * log_det_MA - post * log(q));

gsl_matrix_free(D1);
gsl_matrix_free(DA);
gsl_matrix_free(DAt);
gsl_matrix_free(MA);
gsl_matrix_free(b);
gsl_matrix_free(MA_inv);
gsl_matrix_free(temp);

return (log_marlik);
}

```

Part B: AI Solution

i. Prompt

Follow the problem set pdf file. Write the GSL version of the same marginal likelihood function `marglik` with arguments `gsl_matrix* data`, `int lenA`, and `int* A`. The data matrix has 158 rows and 51 columns, column 1 is the response, and the selected predictors are the columns in `A`. Use GSL matrices and the provided GSL helper functions from `MatricesGSL/`. The code should compute $M_A = I + D_A^T D_A$, the log determinant, the quadratic form $D_1^T D_A M_A^{-1} D_A^T D_1$, and return the log marginal likelihood.

ii. Code Generated

```

double marglik(gsl_matrix *data, int lenA, int *A)
{
    int i, j;
    int n = data->size1;
    gsl_matrix *XA = gsl_matrix_alloc(n, lenA);

    for (i = 0; i < n; i++)
    {
        for (j = 0; j < lenA; j++)
        {
            gsl_matrix_set(XA, i, j, gsl_matrix_get(data, i, A[j] - 1));
        }
    }
}

```

```

gsl_matrix *XAT = transposematrix(XA);
gsl_matrix *MA = gsl_matrix_alloc(lenA, lenA);
matrixproduct(XAT, XA, MA);

for (i = 0; i < lenA; i++)
{
    gsl_matrix_set(MA, i, i, gsl_matrix_get(MA, i, i) + 1.0);
}

double logdetMA = logdet(MA);
double yty = 0.0;
for (i = 0; i < n; i++)
{
    double y = gsl_matrix_get(data, i, 0);
    yty += y * y;
}

gsl_matrix *rhs = gsl_matrix_alloc(lenA, 1);
for (i = 0; i < lenA; i++)
{
    double value = 0.0;
    for (j = 0; j < n; j++)
    {
        value += gsl_matrix_get(XAT, i, j) * gsl_matrix_get(data, j, 0);
    }
    gsl_matrix_set(rhs, i, 0, value);
}

gsl_matrix *MAinv = inverse(MA);
gsl_matrix *sol = gsl_matrix_alloc(lenA, 1);
matrixproduct(MAinv, rhs, sol);

double quad = 0.0;
for (i = 0; i < lenA; i++)
{
    quad += gsl_matrix_get(rhs, i, 0) * gsl_matrix_get(sol, i, 0);
}

double alpha0 = (lenA + 2.0) / 2.0;
double alphaN = (n + lenA + 2.0) / 2.0;
double scale = 1.0 + yty - quad;
double out = lgamma(alphaN) - lgamma(alpha0) - 0.5 * logdetMA
            - alphaN * log(scale);

gsl_matrix_free(XA);
gsl_matrix_free(XAT);
gsl_matrix_free(MA);
gsl_matrix_free(rhs);
gsl_matrix_free(MAinv);
gsl_matrix_free(sol);

return out;
}

```

Part C: Comparison

- **Overall**
 - The AI GSL solution is correct and concise.
 - My solution is more explicit and readable.
- **Correctness**
 - The AI code follows exactly the same GSL-based marginal likelihood calculation as my code.
 - It correctly builds the selected predictor matrix from the columns in **A**.
 - It computes M_A , $\log \det(M_A)$, $D_1^T D_1$, and the quadratic form correctly.
 - The output is also -59.97893 for AI's code
- **Readability**
 - The AI code is clear and compact, seems like AI is better at C/C++ than R.
 - My code is slightly easier to compare with the formula because it creates separate matrices **D1** and **DA**.
 - The AI code avoids creating **D1** and instead accesses the response column directly, which is less readable in my opinion.
- **Data Structures**
 - Both use `gsl_matrix*` for all matrices.
 - Both codes allocate matrices for the selected predictors, transpose, M_A , and temporary vectors.
 - My code has one extra `gsl_matrix*` **D1**, while the AI code uses fewer allocated matrix objects.
- **Algorithms**
 - Both codes use matrix multiplication, matrix inversion, and log determinant helper functions.
 - Both codes explicitly compute the inverse of M_A .